Task 2: Unsupervised Machine Learning

K-Means Clustering Interpretation

Your K-Means algorithm grouped the 48,187 hourly traffic records into 3 distinct operational clusters:

- **Cluster 0 — "Off-Peak Overnight Flow"**
    - Log Metrics: Mean Hour: 3.0 (3:00 AM) | Mean Volume: 857 vehicles/hr | Weather Severity Index: 3.54
    - Interpretation: Represents late-night and early-morning hours when city transit slows down. Lane capacity is heavily underutilized and congestion risk is near zero.
- **Cluster 1 — "Peak Daytime Commute (Clear/Normal Weather)"**
    - Log Metrics: Mean Hour: 15.1 (3:00 PM) | Mean Volume: 4,259 vehicles/hr | Weather Severity Index: 1.70
    - Interpretation: Captures heavy afternoon commuter and business traffic operating under clear or mild weather conditions.
- **Cluster 2 — "High-Volume Traffic Under Severe Weather"**
    - Log Metrics: Mean Hour: 14.0 (2:00 PM) | Mean Volume: 4,096 vehicles/hr | Weather Severity Index: 7.17
    - Interpretation: Isolates high-friction daytime transit windows where heavy traffic volume overlaps with severe atmospheric conditions (e.g., rain, snow, or thunderstorms), representing elevated roadway risk.

Association Rule Mining (Apriori) Interpretation

The Apriori algorithm discretized Hour, Is_Weekend, weather_main, and congestion_category, mining the following top rules with the highest Lift scores:

1. **Rule 1 (Highest Lift = 3.7914)**
   If [‘weather_main_Clouds’, ‘Day_Type_Weekday’, ‘Congestion_Level_Medium’] -> Then [‘Time_of_Day_Night_Late’]

2. **Rule 2 (Lift = 3.7223)**
   If [‘Day_Type_Weekday’, ‘Congestion_Level_Medium’, ‘weather_main_Clear’] -> Then [‘Time_of_Day_Night_Late’]
3. **Rule 3 (Lift = 3.6026)**
   If [‘weather_main_Clouds’, ‘Time_of_Day_Night_Late’] -> Then [‘Day_Type_Weekday’, ‘Congestion_Level_Medium’]

Plain-Language Meaning

Lift measures how much more likely two events are to occur together compared to random chance (where a Lift of 1.0 means no relationship). A Lift of 3.79 indicates that experiencing Medium Congestion on a cloudy weekday is 3.79 times more likely to occur during the late-night transition period (after 6:00 PM as evening rush hour tapers off) than at a random hour of the week. This confirms that weekday evening traffic declines gradually into moderate flow rather than dropping off abruptly.

Task 4: Apply an Advanced AI Technique

1. Why was this technique selected

MLflow was selected because it directly bridges the gap between standalone model experimentation and production MLOps (Machine Learning Operations). In a real-world smart-city deployment, traffic prediction models are continuously retrained on incoming sensor data. Without structured experiment tracking, model hyperparameters, feature versions, and evaluation metrics become scattered across notebooks, leading to irreproducible results and silent performance degradation.

2. How it was implemented

A modular experiment-tracking framework was implemented (Apply_an_Advanced_AI_Technique.py) that logs parameters, performance metrics, and output artifacts across two distinct model architectures into a structured JSON registry (mlflow_model_registry.json) alongside the centralized pipeline.log audit trail:

- **Experiment 1 (Random_Forest_Traffic_Demand_Regression):**
    - Parameters Logged: algorithm="RandomForestRegressor", n_estimators=50, max_depth=12, split_criterion="squared_error".

    - Actual Metrics Logged: Mean_Absolute_Error_MAE = 266.92, R_Squared_Score = 0.9466.
    - Artifact Tracked: engineered_traffic_features.csv.
- **Experiment 2 (LSTM_Deep_Learning_Recurrent_Demand_Prediction):**
    - Parameters Logged: algorithm="LSTM_Recurrent_Neural_Network", input_lookback_window_hours=24, optimizer_engine="Adam", batch_size_limit=128.
    - Actual Metrics Logged: Test_Loss_MSE = 0.00911, True_MAE_Vehicles = 484.61.
    - Artifact Tracked: shap_feature_importance.png.

3. What value it adds

- **Reproducibility & Auditability:** Every model run is stamped with exact hyperparameters and timestamps, allowing the smart-city engineering team to trace which exact configuration produced a given forecast.
- **Objective Model Comparison:** By logging standardized metrics side-by-side, the team can objectively verify that the **Random Forest Regressor** (MAE: 266.92, R^2: 0.9466) outperforms the **LSTM Recurrent Network** (MAE: 484.61) on this tabular hourly dataset while requiring significantly less compute power.
- **Seamless Promotion to Production:** Provides a clean versioned handoff into Task 6's deployment simulation, ensuring only verified "Champion" models reach live API endpoints.

4. Its limitations

- **No Automatic Data Drift Detection:** MLflow tracks static training metrics and parameters, but it cannot detect live data drift once a model is deployed (e.g., if road construction permanently alters traffic flow). It must be paired with an active monitoring layer (as implemented in Task 6).
- **Infrastructure & Storage Overhead:** In full-scale enterprise deployments, hosting a live MLflow tracking server requires dedicated backend databases (such as PostgreSQL) and cloud artifact storage (such as AWS S3), adding ongoing maintenance overhead.

- **Manual Governance Enforcement:** MLflow records experiment history, but it does not automatically block biased or underperforming models from being deployed unless human gatekeeping rules or automated CI/CD checks are enforced around the registry.